

COMP6237 Data Mining

Lecture 11: Topic Modelling (Finding Features II)

Zhiwu Huang

Zhiwu.Huang@soton.ac.uk

Lecturer (Assistant Professor) @ VLC of ECS
University of Southampton

Lecture slides available here:

<http://comp6237.ecs.soton.ac.uk/zh.html>

(Thanks to Prof. Jonathon Hare and Dr. Jo Grundy for providing the lecture materials used to develop the slides.)

Topic Modelling – Problem

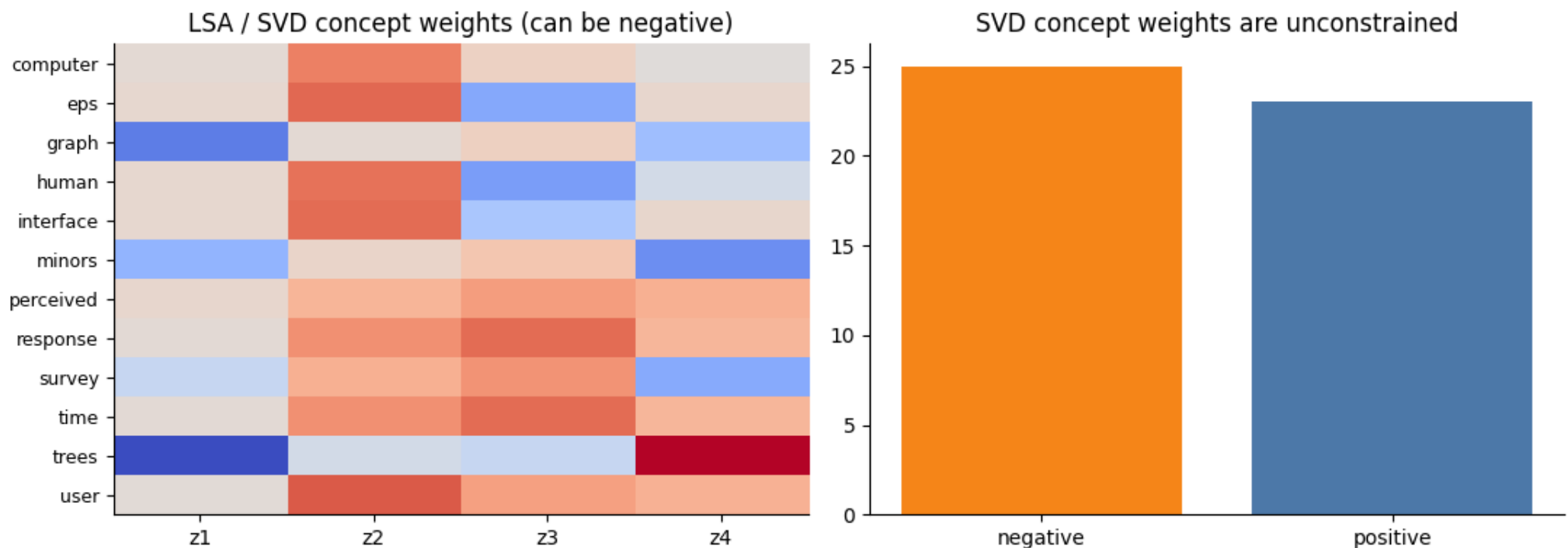
	doc	title
0	m1	Human machine interface for ABC computer applications
1	m2	A survey of user opinion of computer system response time
2	m3	The EPS user interface management system
3	m4	System and human system engineering testing of EPS
4	m5	Relation of user perceived response time to error measurement
5	g1	The generation of random, binary, ordered trees
6	g2	The intersection graph of paths in trees
7	g3	Graph minors IV: Widths of trees and well-quasi-ordering
8	g4	Graph minors: A survey

Topic modeling is an unsupervised document classification method, akin to clustering in numerical data analysis

Topic Modelling – LSA? (recap)

LSA found *concepts* that were linear mixtures of words associated in different documents.

- ▶ Weightings were unconstrained, could be negative
- ▶ Difficult to interpret, couldn't give *meaning* to the concept



Topic Modelling – NMF

Non-negative Matrix Factorisation

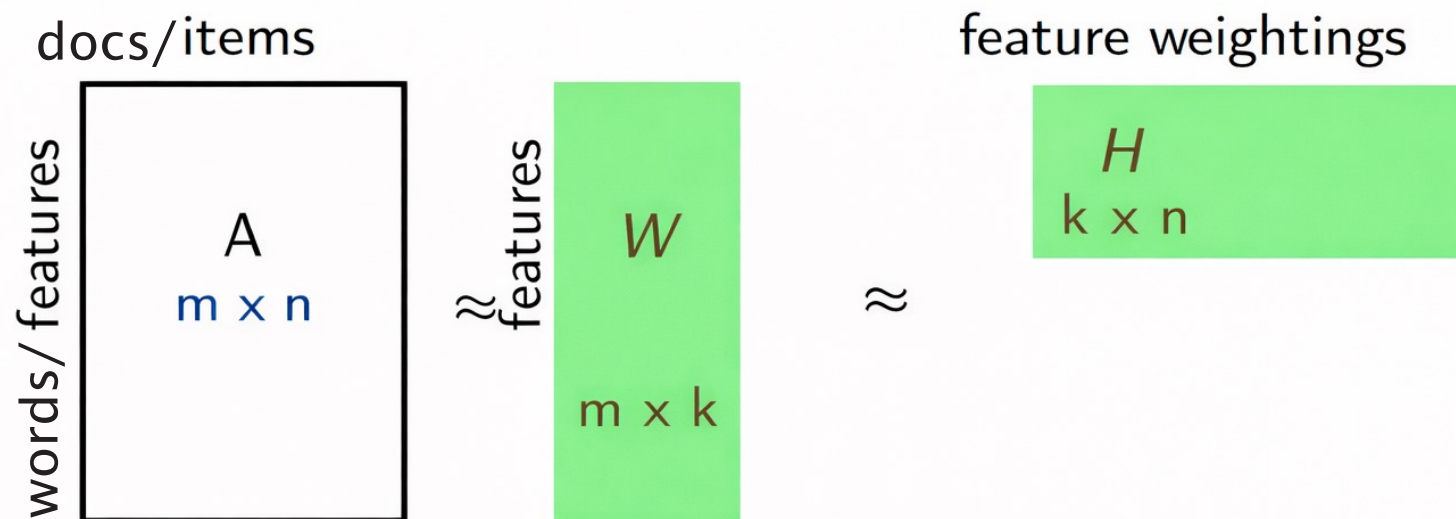
Topic modelling is like clustering as we group documents into similar sets

However we want *soft* clusters
a document should be a weighted mixture of topics

Non-Negative Matrix Factorisation achieves this via a different matrix decomposition

$$A \approx WH$$

Topic Modelling – NMF



- W tells us what the latent topics look like in terms of f words.
- H tells us how much each document belongs to those topics.
- Each column of W is the word mixture for one topic, and each column of H is the topic mixture for one document.
- That is why NMF gives us a soft clustering view:
 - a document does not have to belong to just one topic.
 - Instead, it can belong to several topics with different weights.

Topic Modelling – NMF

W and H are found by an iterative Expectation Maximisation process

A cost function is minimised:

- ▶ Euclidean Norm $\|A - WH\|^2$
- ▶ KL divergence $\sum_{ij} (-A_{ij} \log(WH)_{ij} + (WH)_{ij})$

Lee and Seung 1999, Nature

Topic Modelling – NMF Algorithm

Algorithm 1: NMF Algorithm with Euclidean distance

Data: A ($m \times n$ non-negative matrix), d dimensions to use

Initialise W with $m \times d$ random values;

Initialise H with $d \times n$ random values;

while *not converged* **do**

$$W_{ij} = W_{ij} \frac{(AH^T)_{ij}}{(WHH^T)_{ij}};$$

$$W_{ij} = \frac{W_{ij}}{\sum_k W_{ik}};$$

$$H_{ij} = H_{ij} \frac{W^T A_{ij}}{(WHH^T)_{ij}};$$

end

This has the effect of minimising the norm $\|A - WH\|_F^2$ subject to $W \geq 0, H \geq 0$

Topic Modelling – NMF Algorithm

Algorithm 2: NMF Algorithm with KL-Divergence

Data: A ($m \times n$ non-negative matrix), d dimensions to use

Initialise W with $m \times d$ random values;

Initialise H with $d \times n$ random values;

while *not converged* **do**

$$W_{ij} = W_{ij} \sum_k \frac{A_{ik}}{(WH)_{ik}} H_{jk};$$

$$W_{ij} = \frac{W_{ij}}{\sum_k W_{ik}};$$

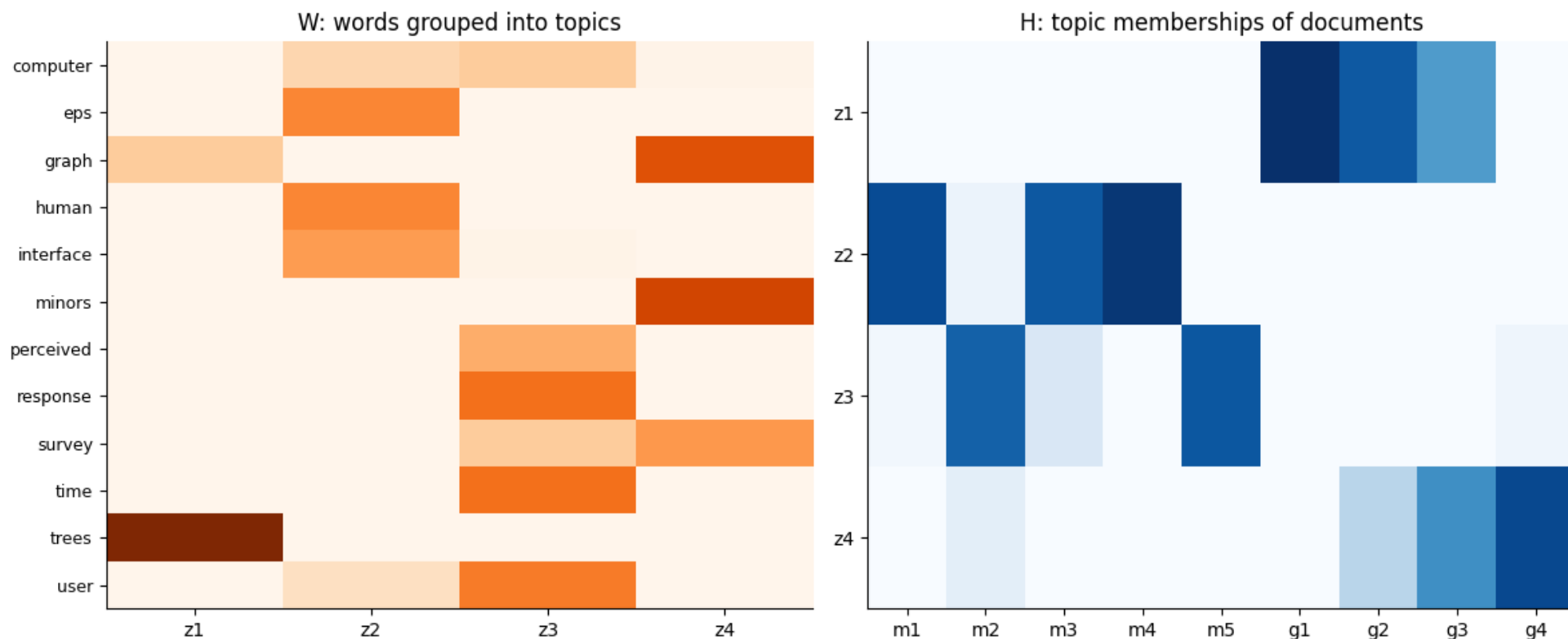
$$H_{ij} = H_{ij} \sum_k W_{ki} \frac{A_{kj}}{(WH)_{kj}};$$

end

This has the effect of minimising the generalized KL Divergence

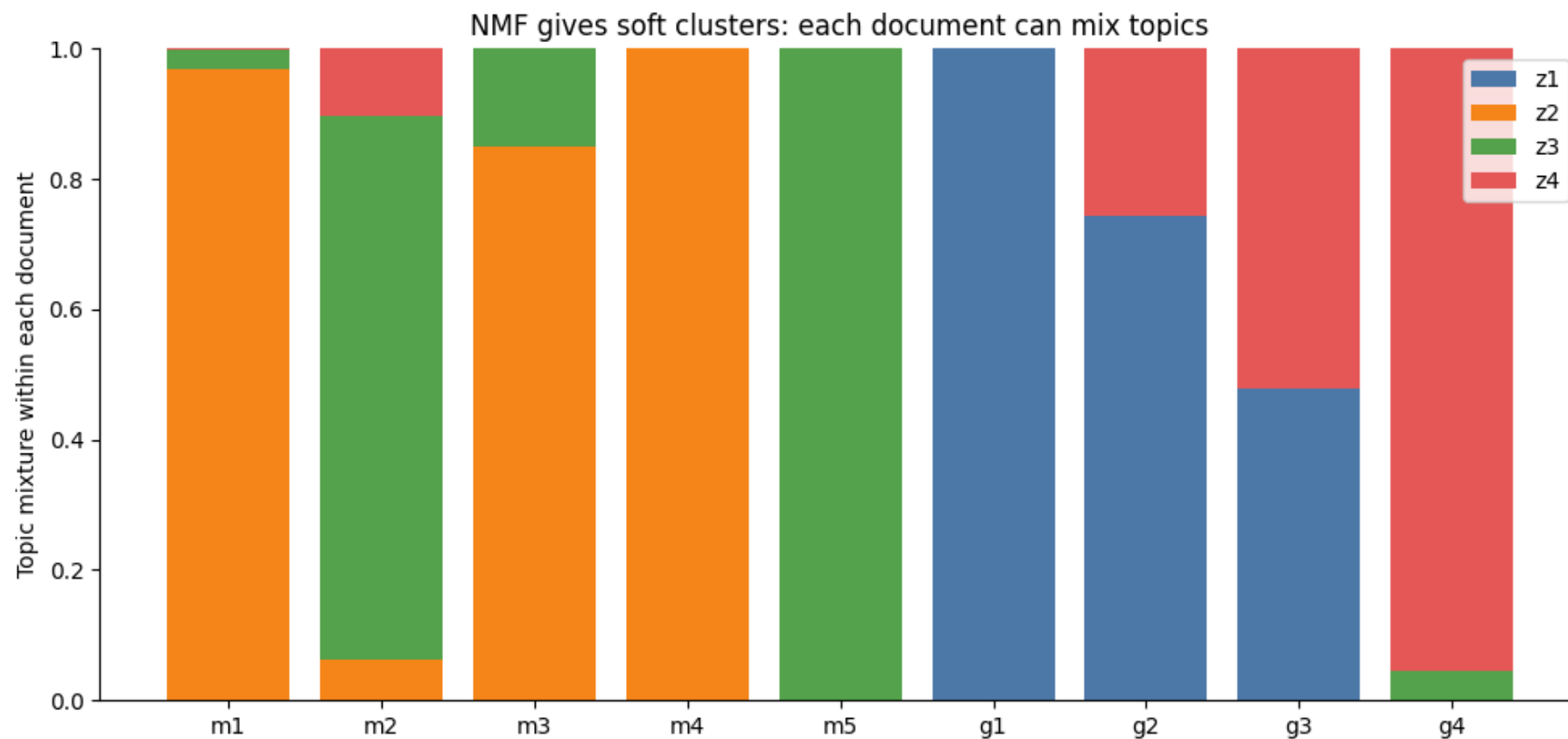
$$\sum_{ij} (-A_{ij} \log(WH)_{ij} + (WH)_{ij}) \text{ subject to } W \geq 0, H \geq 0$$

Topic Modelling – NMF Examples



W has the *basis vectors*, showing how the words are clustered
 H has the topic memberships for the documents.

Topic Modelling – NMF Examples



Topic Modelling – NMF Algorithm

For this method, and for LSA, the size of the reduced dimensionality is chosen manually

Can chose based on the error from reconstruction though like K means, and K nearest neighbours, this will be lower for higher values of d

Can run many times and build up a *consensus matrix*

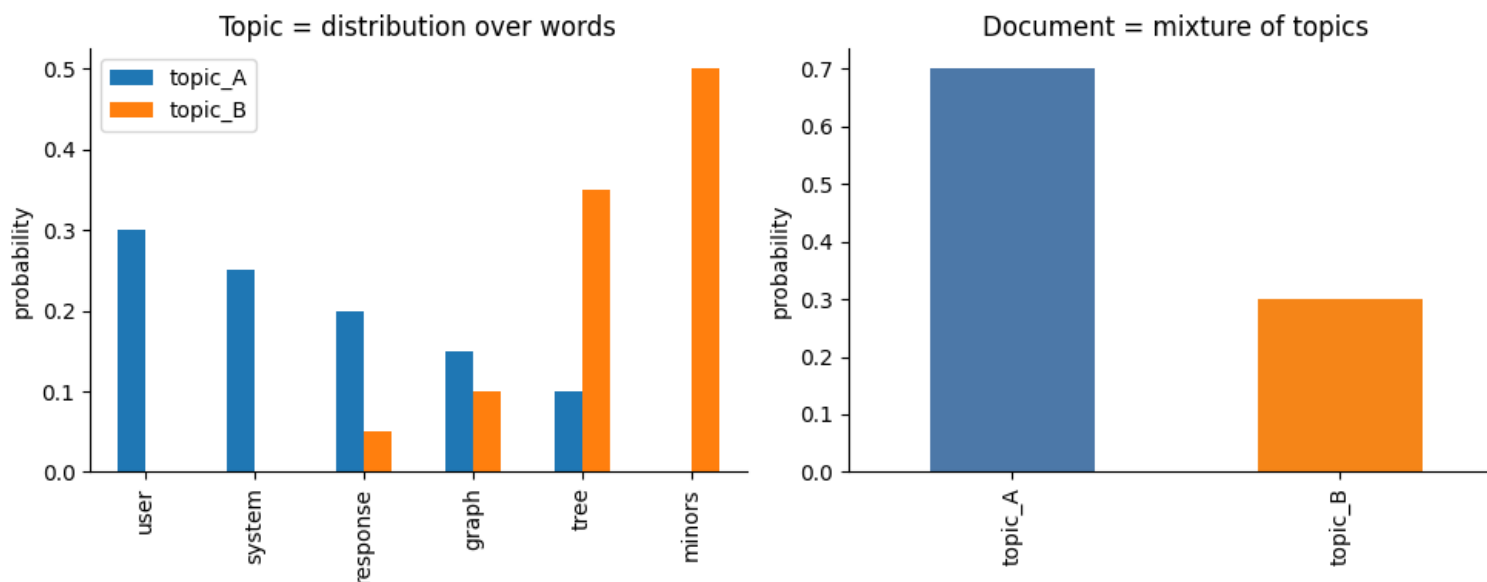
Can also examine the *stability* of multiple random initialised runs for each value of d

[Demo: Finding_Features_Visual](#)

Topic Modelling – Probabilistic Models

We model a **document** as a mixture of topics

A **topic** is a distribution over words



Generative idea

To generate each word in a document:

1. **Choose a topic** according to the document's topic distribution
2. **Choose a word** according to that topic's word distribution

Topic Modelling – PLSA

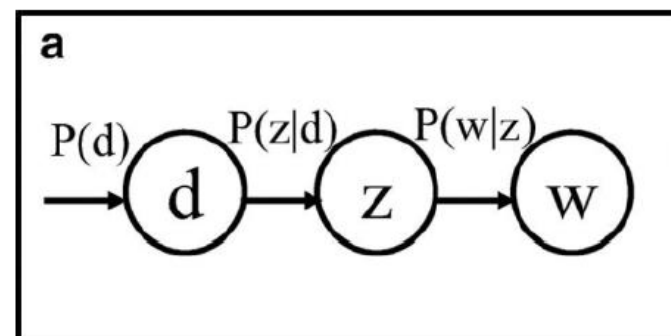
- **Basic Generative Model**

- Select document d with probability $P(d)$
- Select a latent class z with probability $P(z|d)$
- Generate a word w with probability $P(w|z)$

- **Joint Probability Model**

$$P(d, w) = P(d)P(w|d)$$

$$= P(d) \sum_{z \in \mathbb{Z}} P(w|z)P(z|d) \quad \text{PLSA models co-occurrence}$$



Generative idea

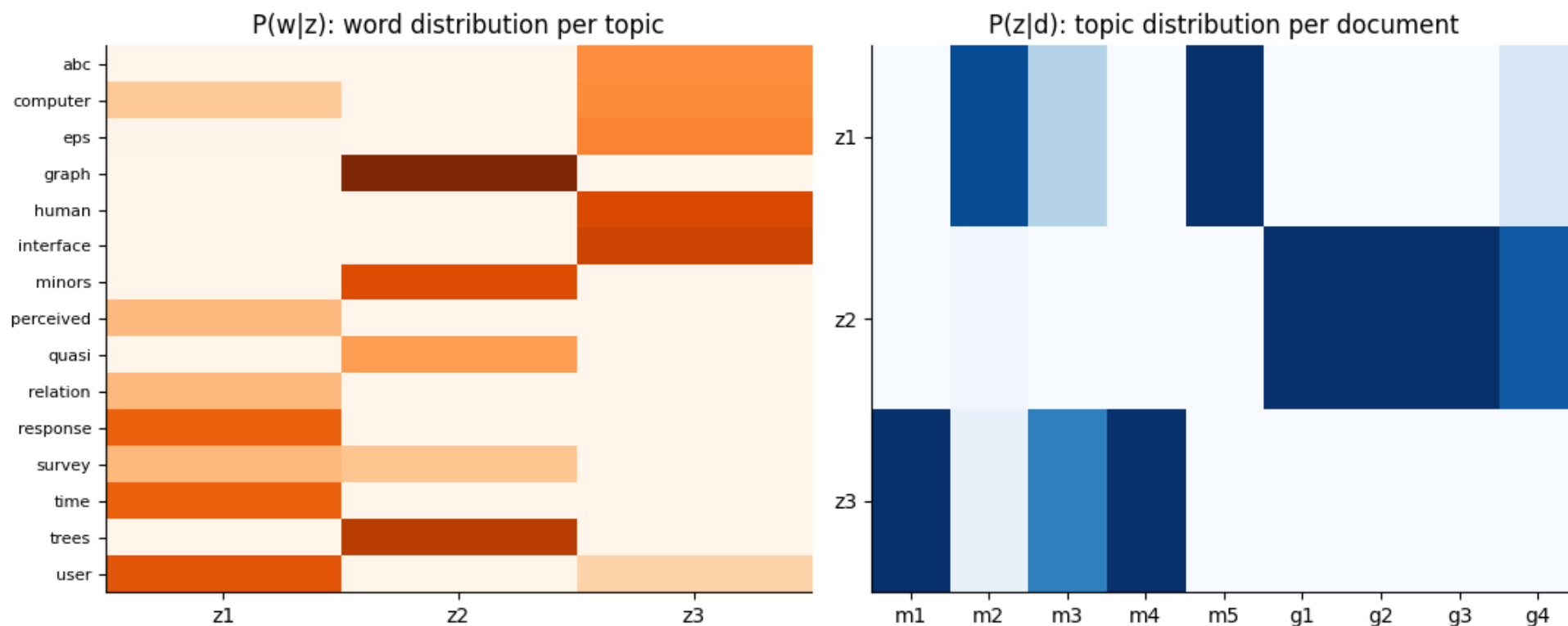
To generate each word in a document:

$$P(w|d) = \sum_{z \in \mathbb{Z}} P(w|z)P(z|d)$$

1. **Choose a topic** according to the document's topic distribution
2. **Choose a word** according to that topic's word distribution

$P(d)$ can be determined directly from our corpus. $P(w|z)$, $P(z|d)$ are modeled as multinomial distributions, and can be trained using the [expectation-maximization](#) (EM) algorithm

Topic Modelling – PLSA



All the probabilities are non-negative! => Interpretable

Topic Modelling – PLSA

Inference Process

Now a new document arrives.

We already have the learned topics $P(w \mid z)$, but this new document was not in training, so it does **not** already have a $P(z \mid d)$.

So for test time, PLSA does this:

- keep the learned $P(w \mid z)$ fixed
- create a new topic mixture for the new document
- iteratively fit that new $P(z \mid d_{\text{new}})$ to explain the document words

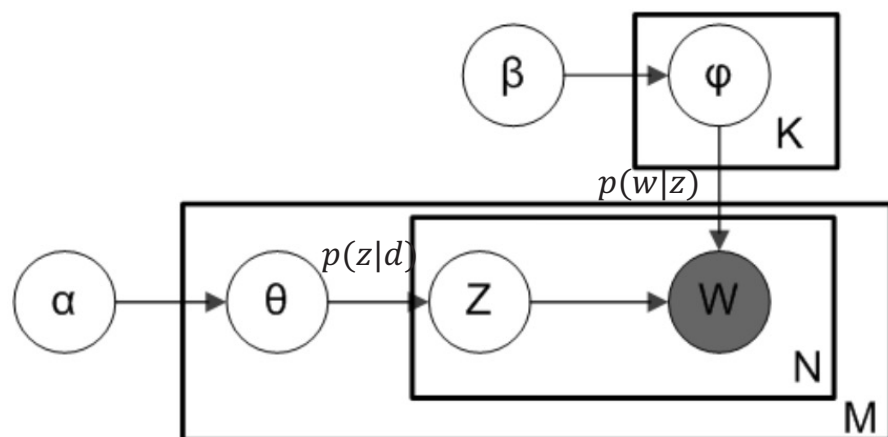
So PLSA test-time prediction is basically:

freeze the topics, then fit a fresh document-topic mixture for the new document.

That is why PLSA feels a bit awkward for new documents: every new document needs its **own extra fitting step**.

[Demo: Finding_Features_Visual](#)

Topic Modelling – LDA



Latent Dirichlet Allocation (LDA) - also in AML
Bayesian Extension to PLSA

- ▶ Uses a Dirichlet prior on the topic distribution per document
- ▶ Fully Generative
- ▶ Bayesian Inference to learn parameters
- ▶ Better than PLSA for small datasets, otherwise similar

The Dirichlet priors **regularize** the estimation:

- α controls **how many topics appear per document**
- β controls **how many words dominate a topic**

LDA describes how documents **could be generated**:

1. Draw topic–word distributions

$$\phi_z \sim \text{Dir}(\beta)$$

2. For each document d :

- draw topic mixture

$$\theta_d \sim \text{Dir}(\alpha)$$

3. For each word position n :

- choose topic

$$z_{dn} \sim \text{Mult}(\theta_d)$$

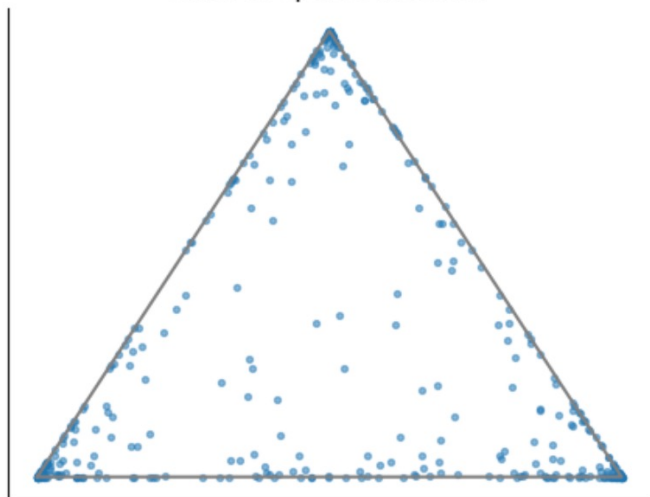
- generate word

$$w_{dn} \sim \text{Mult}(\phi_{z_{dn}})$$

Topic Modelling – LDA

Dirichlet prior controls what topic mixtures look like

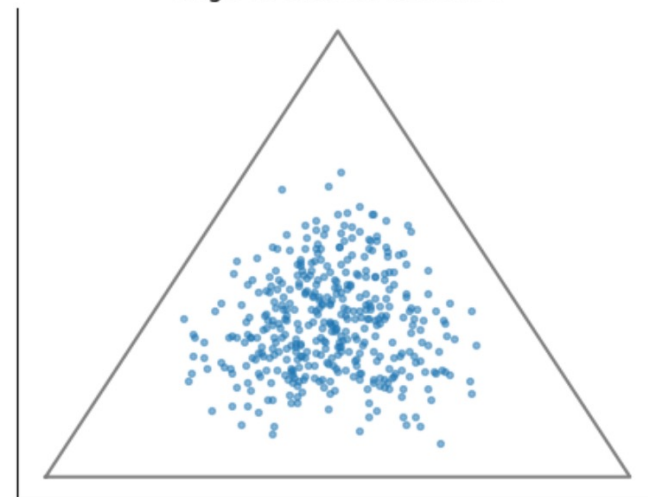
small α : sparse mixtures



$\alpha = 1$: uniform-ish



large α : smooth mixtures



[Demo: Finding_Features_Visual](#)

Topic Modelling – LDA

LDA describes how documents **could be generated**:

1. Draw topic–word distributions

$$\phi_z \sim \text{Dir}(\beta)$$

2. For each document d :

- draw topic mixture

$$\theta_d \sim \text{Dir}(\alpha)$$

3. For each word position n :

- choose topic

$$z_{dn} \sim \text{Mult}(\theta_d)$$

- generate word

$$w_{dn} \sim \text{Mult}(\phi_{z_{dn}})$$

From the corpus we observe only **words** w .

But the LDA model contains hidden variables:

- z : topic assignment for each word
- θ_d : topic mixture for each document
- ϕ_z : word distribution for each topic

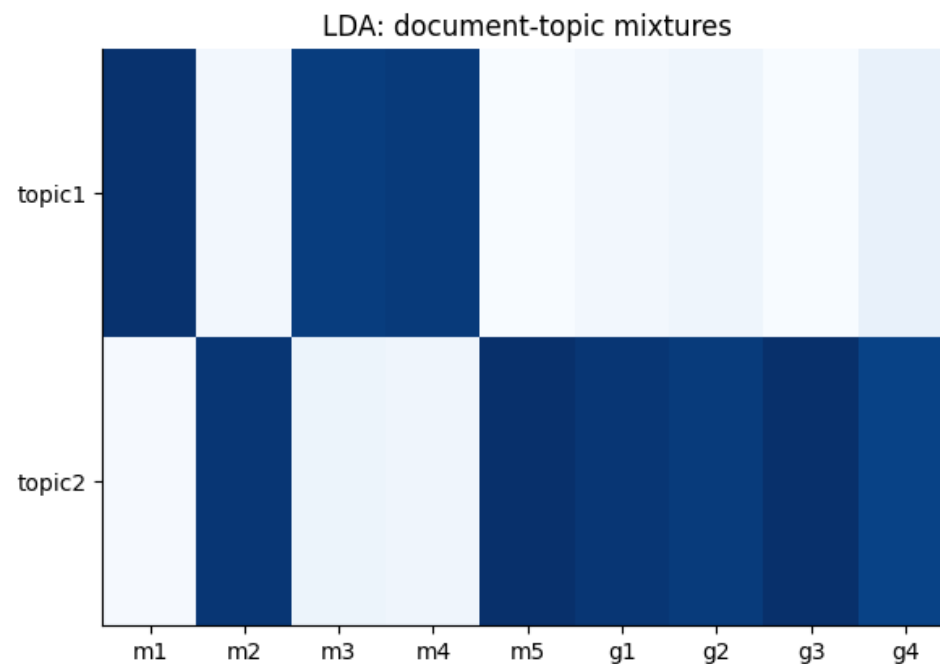
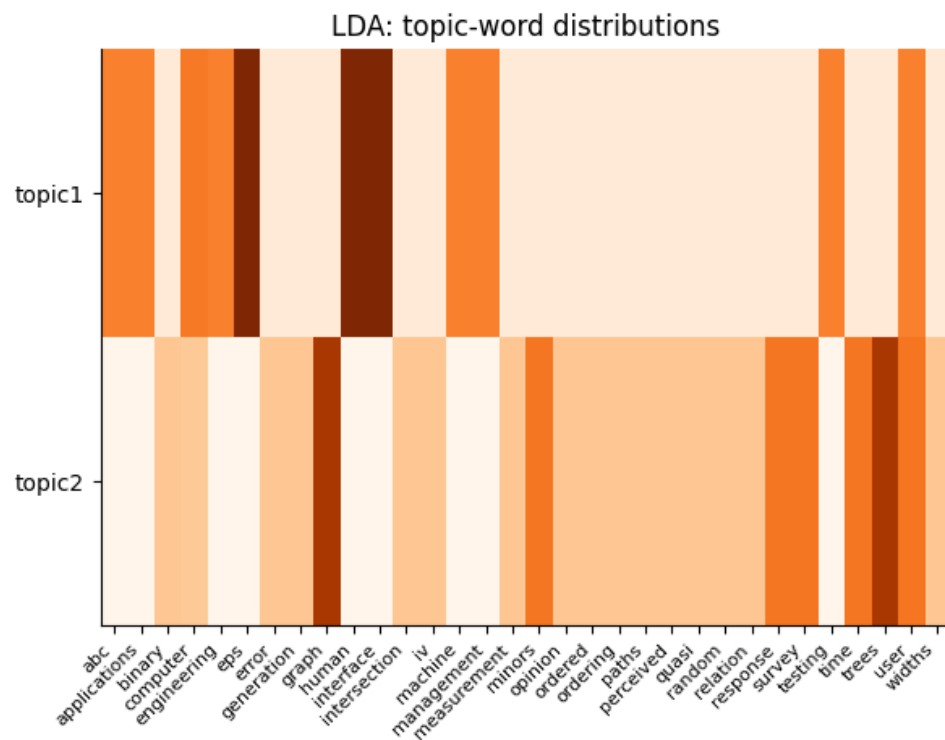
Ideally we want the **posterior distribution**:

$$P(\theta, \phi, z \mid w, \alpha, \beta)$$

Bayesian Inference to learn these parameters:

Approximate the posterior distribution with a simpler distribution and optimize it. Used in the original LDA paper.

Topic Modelling – LDA



All the probabilities are non-negative! => Interpretable

Topic Modelling – LDA

At test time, **LDA does not rebuild the whole model.**

[Demo: Finding_Features_Visual](#)

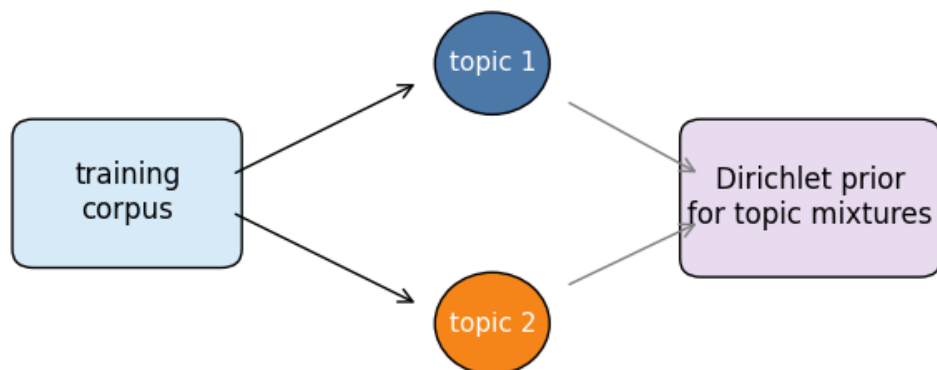
Training has already learned:

- the **topic–word distributions**: what each topic looks like
- the **Dirichlet prior**: what document-topic mixtures usually look like

So for a new document, the only missing part is its **document-topic mixture**.

In other words:

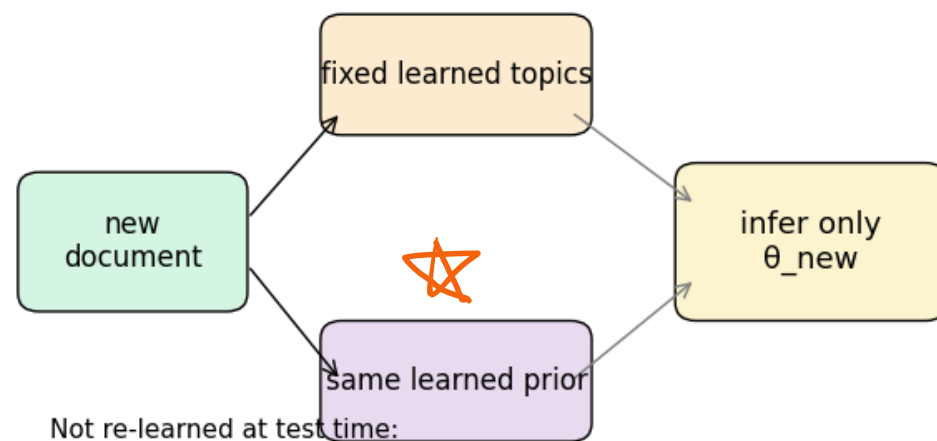
- **training** learns the topic system
- **testing** just places the new document inside that topic system



Learned and kept for later:

- topic-word distributions
- prior / typical mixture behaviour

Training: learn the topic system



Not re-learned at test time:

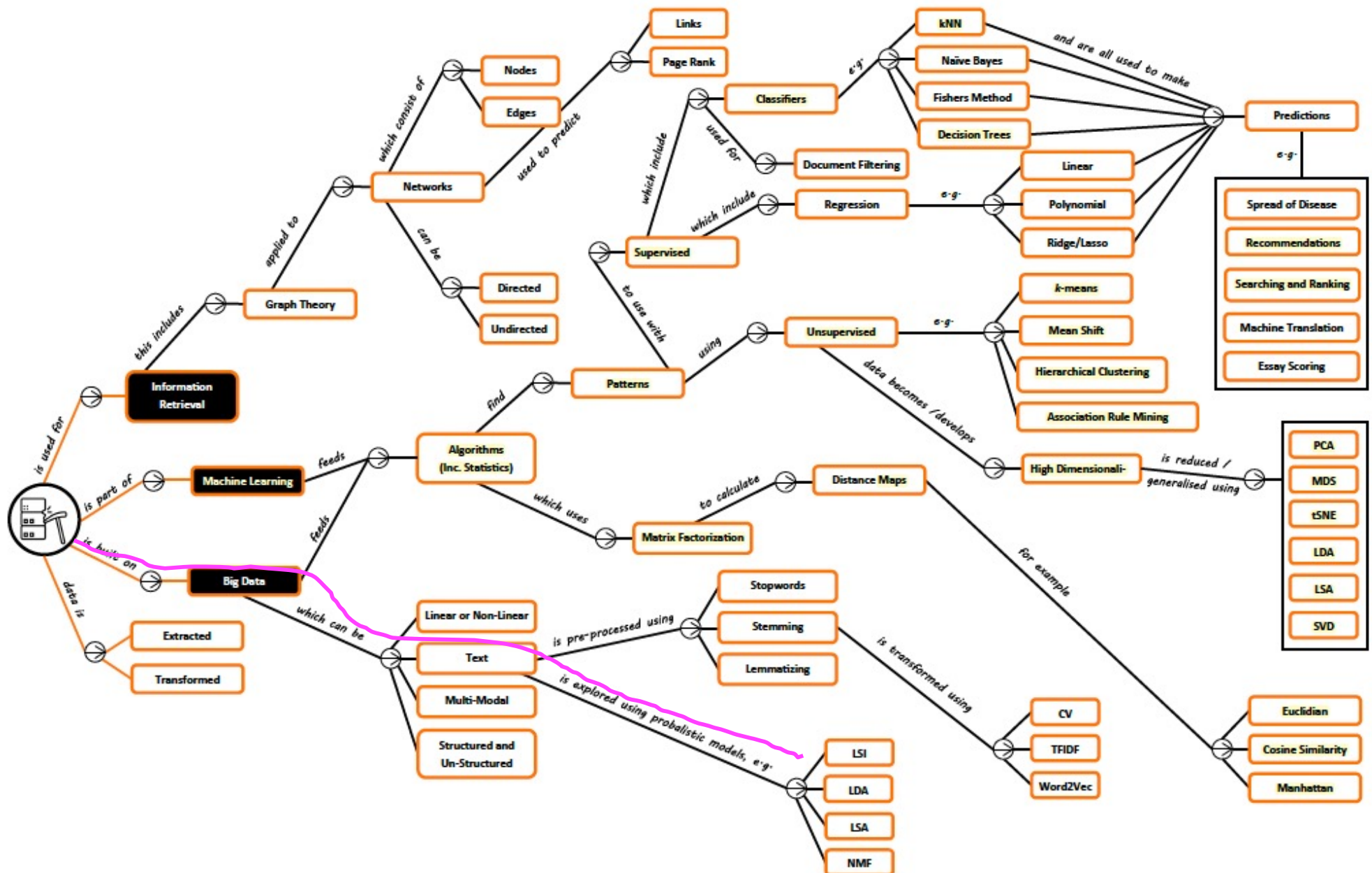
- topics
- global word-topic structure
- prior behaviour

Testing: infer only the new document mixture

Topic Modelling – Summary

- Topic modelling is unsupervised document clustering / soft clustering.
- LSA uses linear algebra, but concepts can have negative weights and be hard to interpret.
- NMF gives non-negative topics, which are easier to read as parts-based topics.
- PLSA introduces hidden topics probabilistically, but new documents needs extra fitting step.
- LDA adds Dirichlet priors, giving better generalization to new documents.

Topic Modelling – Roadmap



Topic Modelling – Textbook

CHAPTER 10

Finding Independent Features

Most of the chapters so far have focused primarily on *supervised* classifiers, except Chapter 3, which was about *unsupervised* techniques called *clustering*. This chapter will look at ways to extract the important underlying features from sets of data that are not labeled with specific outcomes. Like clustering, these methods do not seek to make predictions as much as they try to characterize the data and tell you interesting things about it.

- ▶ Programming Collective Intelligence: Building Smart Web 2.0 Applications *T. Segaran*.

Topic Modelling – Learning Outcomes

- **LO1:** Demonstrate an understanding of techniques for finding independent features for topic modeling, such as: (exam)
 - ❖ Comprehending the core concepts of NMF and apply NMF on a dataset
 - ❖ Understanding the key idea and steps of probabilistic models like PLSA and LDA
 - ❖ Discussing the advantages and disadvantages of the learned algorithms
- **LO2:** Implement the learned algorithms for topic modeling (coursework)

Assessment hints: Multi-choice Questions (single answer: concepts, calculation etc)

- *Textbook Exercises: textbooks (Programming + Mining)*
- *Other Exercises: <https://www-users.cse.umn.edu/~kumar001/dmbook/sol.pdf>*
- *ChatGPT or other AI-based techs*